

Optimising Weak Reference Processing in OpenJDK's ZGC

Master's Thesis in Computer Science

Fredrik Hammarberg

MSc Computer and Information Engineering, Uppsala University

fredrik.hammarberg00@outlook.com

May 2026

Master's Thesis Defence – Uppsala University



UPPSALA
UNIVERSITET

Agenda

- 1 Background & Motivation
- 2 Design & Implementation
- 3 Evaluation
- 4 Results & Analysis
- 5 Discussion & Conclusion



Background & Motivation

GC Performance in Managed Runtimes

- ▶ GC pauses affect application latency; concurrent GC work competes with application threads for CPU and memory bandwidth
- ▶ **ZGC** achieves sub-millisecond pauses independent of heap size; marking, reference processing, and relocation are mostly concurrent
- ▶ One underoptimised component: **weak reference processing**
 - cost scales linearly with the number of `WeakReference` objects

Motivation

ZGC's reference-processing pipeline treats all weak references uniformly, even those that do not need the callback mechanism it provides.

Weak References in Java

- ▶ `WeakReference<T>` allows objects to be reclaimed despite being referenced
- ▶ Optional `ReferenceQueue`: callback when referent is collected
- ▶ **Without a queue** (common case): pure weak semantics, no GC callback needed
- ▶ Widely used in `WeakHashMap`, caches, observer patterns

Without queue

```
new WeakReference<>(obj)
```

Only weak reachability required

With queue

```
new WeakReference<>(obj, q)
```

GC notification when `obj` is cleared

ZGC's Reference-Processing Pipeline

- ➊ **Discovery** – reference objects appended to per-worker *intrusive linked lists* via hidden `discovered` field
- ➋ **Processing** – check referent liveness; clear if unreachable
- ➌ **Enqueue** – cleared references put on the *pending list*; `ReferenceHandler` delivers them to their `ReferenceQueue`

The Problem

Steps 1–3 are applied *uniformly* to all weak references – even queue-less ones that will never be enqueued. Wasted traversal and linked-list overhead for the most common case.

Research Questions

RQ1 – Pipeline Optimisations

How do targeted modifications to ZGC's processing of `WeakReference` objects *without* a `ReferenceQueue` affect reference-processing time, total GC cycle time, GC memory, and Java heap usage?

RQ2 – Weak Fields

How does expressing weak semantics through *annotated object fields* affect the same metrics compared to the optimised `WeakReference` pipeline approach?

Design & Implementation

Four Mechanisms, Eight Variants

Three pipeline optimisations for queue-less

`WeakReference`:

- ▶ **sep** – skip-enqueue separation
- ▶ **dyn** – dynamic array replacing the linked list
- ▶ **clear_path** – specialised clear path

One structural alternative:

- ▶ **weak_fields** – `@weak` field annotation

All $2^3 = 8$ combinations benchmarked independently.

Variant	cp	sep	dyn
none	–	–	–
dyn	–	–	✓
sep	–	✓	–
sep_dyn	–	✓	✓
cp	✓	–	–
cp_dyn	✓	–	✓
cp_sep	✓	✓	–
all	✓	✓	✓

Mechanism 1: Skip-Enqueue Separation

Idea: Route queue-less weak refs to a separate discovered list – bypasses the pending list and ReferenceHandler thread entirely.

Listing 1: Discovery split – pseudocode

```
1 bool discover_reference(oop ref, ReferenceType type) {  
2     if (!should_discover(ref, type)) return false,  
3  
4     if (type == REF_WEAK && !has_reference_queue(ref)) {  
5         // Queue-less: no-enqueue list, never touches ReferenceHandler  
6         no_queue_list_per_worker.append(ref),  
7     } else {  
8         // Queue-backed and all other types: standard pipeline  
9         discovered_list_per_worker.append(ref),  
10    }  
11    mark_discovered(ref), return true,  
12 }
```

- ▶ Each GC worker has its own list – no inter-thread contention
- ▶ Queue-less list processed by a specialised loop that never enqueues

Mechanism 2: Dynamic Array

Idea: Replace the intrusive linked list with a contiguous `ZWeakRefArray` for cache-friendly traversal and pre-loaded referent data.

Listing 2: Array-based processing – pseudocode

```
1 void process_no_queue_refs(ZWeakRefArray& arr) {  
2     size_t kept = 0,  
3     for (size_t i = 0, i < arr.length(), i++) {  
4         const ZWeakRefData& d = arr.at(i), // sequential: cache-friendly  
5         d.mark_not_discovered(),  
6         if (try_make_inactive(d)) { cleared++, }  
7         else { kept++, }  
8     }  
9     arr.clear_and_reserve(kept), // retain capacity for next cycle  
10 }
```

- ▶ Array grows on the C heap; amortised $O(1)$ append per discovery
- ▶ Referent address and value *pre-loaded* during discovery

Mechanism 3: Specialised Clear Path

Idea: For old-generation queue-less `WeakReference` objects, replace the CAS-based clear with a plain store.

Standard clear path

1. Load barrier on referent
2. CAS: referent $\rightarrow$ coloured null

Specialised clear path

1. Use pre-loaded value (no barrier)
2. Direct store (no CAS)

Why safe?

In the relevant STW pause, no application thread can update the referent of an old-generation queue-less reference – only the GC can. The CAS is therefore unnecessary.

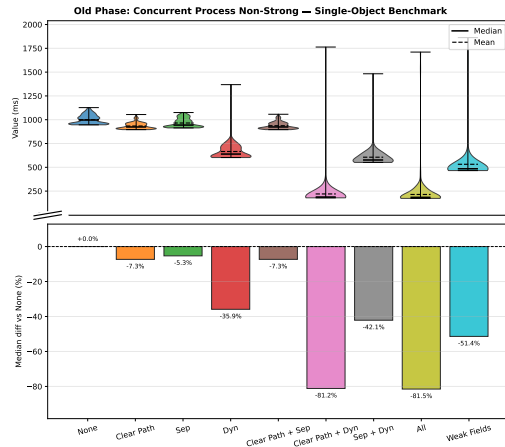
Superadditive Interaction: dyn + clear_path

- ▶ **dyn alone:** -36% (cache locality, but still CAS)
- ▶ **clear_path alone:** -7% (faster clear, still linked list)
- ▶ **dyn + clear_path:** -81%

Why superadditive?

dyn pre-loads referent data during discovery.

clear_path uses pre-loaded data – removing the load barrier *and* the pointer-chase simultaneously.



Non-strong time (single-object)

Mechanism 4: Weak Fields

Idea: Weakness as a *field property* – no `WeakReference` wrapper object required.

Usage

```
import java.lang.ref.weak;  class Holder { @weak Target ref; }
```

- ▶ Annotation recognised by the class-file parser (ZGC-only; 27 files across compiler, interpreter, JIT, GC)
- ▶ Fields discovered during old-generation marking; stored in per-worker `ZWeakFieldArray`
- ▶ Unreachable referent: annotated field zeroed directly – no pending list, no `ReferenceHandler`
- ▶ **Key:** no wrapper objects allocated or promoted

Evaluation

Benchmark Design

Single-object

- ▶ 20M holder objects pointing to one shared target
- ▶ Target released; `System.gc()` triggered
- ▶ Maximises per-cycle reference-processing load

Multi-object

- ▶ 2M objects, random-size payloads
- ▶ 5 rounds: 20% of strong refs released per round
- ▶ More realistic liveness and locality patterns

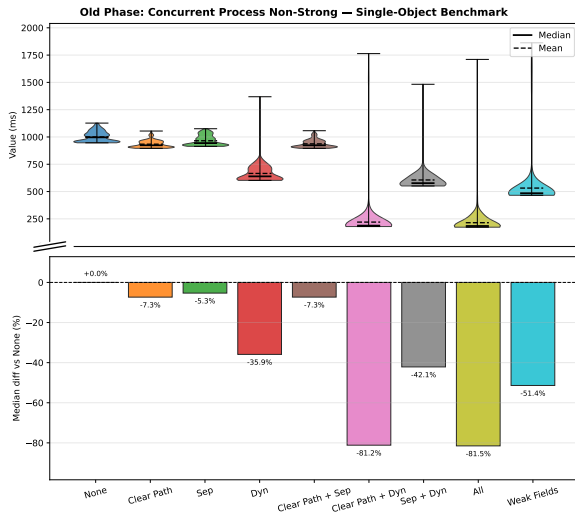
Execution Environment

UPPMAX supercomputer (SLURM), 48-core nodes, 100 GB heap, `-XX:+UseZGC`, 250 outer iterations, 4 instances pinned to isolated CPU sets.

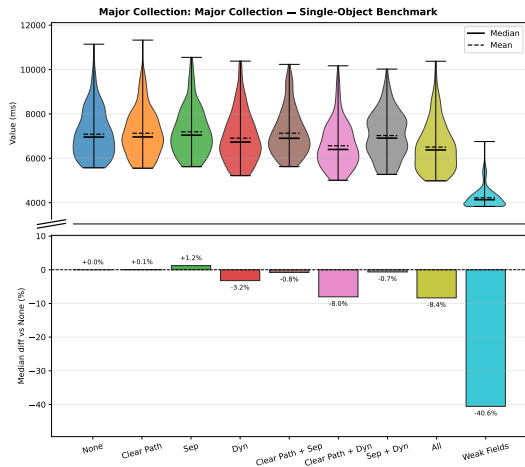
Results & Analysis

Non-Strong Processing Time

Variant	Median (ms)
none	996.9
sep_only	943.8 (-5%)
cp_only	923.7 (-7%)
dyn_only	639.1 (-36%)
sep_dyn	576.9 (-42%)
cp_dyn	187.9 (-81%)
all	184.4 (-81%)
wf	484.6 (-51%)



Major Collection Time – The Ceiling



Variant	Median (ms)
---------	-------------

none	6,958
------	-------

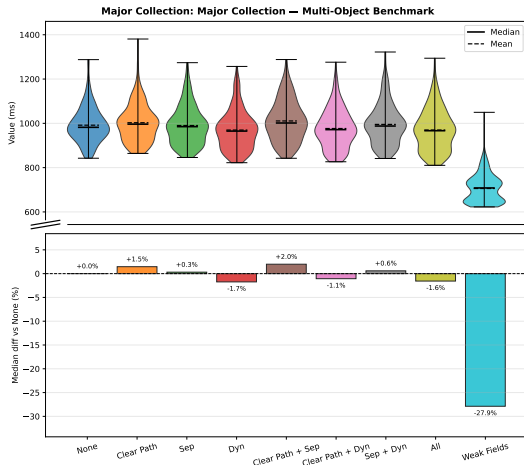
all	6,377 (-8%)
-----	-------------

wf	4,136 (-41%)
----	--------------

Why only 8%?

Non-strong processing = **14.3%** of total major GC time. Even 81% reduction there moves the needle by at most $\approx 11\%$.

Multi-Object Benchmark



- ▶ Non-strong = only **4.5%** of baseline major GC time in this benchmark
- ▶ All pipeline variants: 965–1,002 ms vs. 982 ms baseline – distributions fully overlap
- ▶ `weak_fields`: 708 ms (–28%)

Pattern

Pipeline improvements bounded by the fraction they address. `weak_fields` benefits every phase by reducing the object population.

Memory Trade-offs (Single-Object)

Auxiliary GC memory:

Variant	MB
none	120
dyn	308 (×2.6)
sep_dyn	308 (×2.6)
cp_dyn	1,268 (×10.6)
all	884 (×7.4)
wf	446 (×3.7)

- ▶ `cp_dyn`: large entry struct (referent address + value pre-loaded)
- ▶ `all` saves 30% vs `cp_dyn`: `sep` eliminates the reference-address field
- ▶ `weak_fields` Java heap: **806 MB** vs **1,720 MB** (−53%) – no wrappers

Structural vs. Algorithmic Advantage

Pipeline optimisations

- ▶ Accelerate processing of existing `WeakReference` objects
- ▶ Improvements confined to non-strong processing phase

Weak fields

- ▶ Eliminate `WeakReference` objects entirely
- ▶ Fewer live objects $\Rightarrow$ less work in every phase:
marking (−31%), young-gen (−44%)

Single-object benchmark

Concurrent mark:

none: 2,575 ms

wf: 1,782 ms (−31%)

Young-gen (Promote All):

none: 3,201 ms

wf: 1,784 ms (−44%)

Discussion & Conclusion

Comparison with Other Languages

Language	Mechanism	Callback?
Java	<code>WeakReference</code> + <code>ReferenceQueue</code>	Optional
Python	<code>weakref.ref</code> + <code>callable</code>	Optional
Go	<code>weak.Pointer</code>	No
C++	<code>std::weak_ptr</code>	No
.NET	<code>WeakReference<T></code>	No
Lua	Weak tables (field-level)	No

- ▶ Most languages treat weak reachability and lifecycle notification as separate concerns
- ▶ Java couples both into one object and one pipeline
- ▶ `@weak` brings Java closer to Lua's field-level model – weakness as a structural property of the field, not a wrapper

Conclusions

RQ1 – Pipeline Optimisations

- ▶ `cp_dyn` and `all`: **81%** reduction in non-strong processing time (superadditive interaction)
- ▶ Major collection improvement: at most **8%** – bounded by the 14.3% fraction non-strong processing occupies

RQ2 – Weak Fields

- ▶ **41%** major GC time reduction; **53%** heap reduction
- ▶ Structural advantage: fewer objects in every GC phase

Overall Finding

Weak-reference overhead is a **representation problem**, not a pipeline problem. Meaningful improvement requires rethinking how weak semantics is encoded.

Future Work

- 1 **Production workloads** – Java Flight Recorder on real applications with diverse reference patterns
- 2 **ReferenceHandler load** – latency impact for queue-backed references in mixed (queue-backed + queue-less) workloads
- 3 **Extend `clear_path`** – to `SoftReference`, `PhantomReference`, and queue-backed `WeakReference`
- 4 **Upstream integration** – submit via JDK-8029205
- 5 **Harden `@weak`** – test suite, formal correctness, upstream submission

References I

[◀ Back to start](#)

- [1] OpenJDK, “Jep 333: Zgc: A scalable low-latency garbage collector (experimental).” <https://openjdk.org/jeps/333>, 2018.
Accessed: 2026-03-12.
- [2] OpenJDK, “Jep 439: Generational zgc.” <https://openjdk.org/jeps/439>, 2023.
Accessed: 2026-03-12.
- [3] E. Österlund, “JVMLS – generational ZGC and beyond.” <https://inside.java/2023/08/31/generational-zgc-and-beyond/>, 2023.
Accessed: 2026-05-25.

References II

- [4] Oracle, “java.lang.ref (java se 25).”
<https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/ref/package-summary.html>, 2025.
Accessed: 2026-03-12.
- [5] OpenJDK Bug System, “Unnecessary pending-list traversal for references without referencequeue.”
<https://bugs.openjdk.org/browse/JDK-8029205>, n.d.
Accessed: 2026-02-24.
- [6] U. Drepper, “What every programmer should know about memory,” tech. rep., Red Hat, Inc., 2007.
Accessed: 2026-04-07.

References III

- [7] R. Jones, A. Hosking, and E. Moss, *The Garbage Collection Handbook: The Art of Automatic Memory Management*. Chapman and Hall/CRC, 2011.
- [8] H. Lieberman and C. Hewitt, “A real time garbage collector based on the lifetimes of objects,” Tech. Rep. AIM-569a, MIT Artificial Intelligence Laboratory, Oct. 1981.
Accessed: 2026-02-24.